

Università degli Studi

Corso di Laurea in Informatica

Insegnamento: Introduzione alla Programmazione per il Web

Relazione di Progetto

GymBros Centre Web App

Anno Accademico 2025–2026

ID Team	07
Componenti	Stefano Videsott Alessandro Como

Repository del progetto disponibile a
https://github.com/StefanoVidesott/project_web.

1. Introduzione

1.1. Obiettivo del progetto

L'obiettivo del progetto è realizzare una web app per il centro fitness e benessere *GymBros Centre*, che permetta ai clienti e al personale amministrativo di gestire abbonamenti e sessioni di allenamento. L'applicazione supporta quattro ruoli: amministratore, utente Prova, utente Basic e utente Pro, ciascuno con funzionalità diverse in base al piano sottoscritto.

1.2. Scelte architetturali

Come base abbiamo usato l'esempio fornitoci della professoressa (`esempio_security`) per impostare il pattern MVC con Spring Boot, Spring Security e Thymeleaf. Da lì abbiamo applicato in tutta la webapp principale il pattern **controller** → **service** → **repository**: ogni controller delega la logica ai service dedicati, che accedono ai dati tramite repository. Separare questi tre strati consente di modificare ciascuno senza toccare gli altri.

1.3. Soluzioni alternative considerate

Conteggio degli allenamenti completati. Inizialmente avevamo una tabella separata come log, condivisa tra programmi default e personalizzati. Rileggendo la consegna ci siamo resi conto che non era consentito; abbiamo quindi usato quattro colonne in `USERDATA` per i programmi predefiniti e una tabella `CUSTOM_TRAININGS_COUNTER` per quelli custom.

Selezione del piano in fase di registrazione. La selezione del piano avveniva tramite dropdown. Per mostrare le informazioni descrittive e i costi di ciascun piano lo abbiamo sostituito con un set di *card* dotate di radio button.

Endpoint aggiuntivo nel REST service. Per la creazione di programmi personalizzati abbiamo aggiunto `GET /exercise-names`, che restituisce solo i nomi degli esercizi per consentire la selezione tramite dropdown ed evitare errori di battitura.

Navbar duale. Inizialmente avevamo due componenti navbar distinti. La libreria `thymeleaf-extras-springsecurity6` ha risolto il problema: con `sec:authorize="isAnonymous()"` e `sec:authorize="isAuthenticated()"` si gestisce la visibilità di ogni elemento nel template con un solo componente.

Schema del database del REST service. La struttura del database ha subito più revisioni. Aggiungere un identificatore numerico ai programmi predefiniti ha

semplificato il collegamento con i contatori in `USERDATA` e l'implementazione delle statistiche per tipo di allenamento.

2. Architettura

2.1. Pattern MVC e organizzazione del progetto

Abbiamo adottato il pattern architetturale MVC (Model–View–Controller), strutturato su tre livelli distinti.

- **Model.** I POJO del dominio (`User`, `Training`, `Composition`, `CustomComposition`, `Review`, ...) e i repository gestiscono l'accesso al database H2 tramite `JdbcTemplate`. Mapper dedicati (`UserMapper`, `TrainingMapper`, `ExerciseMapper`, ...) si occupano della conversione da `ResultSet` a oggetto del dominio, tenendo questa logica fuori dai repository. La gestione degli utenti di sicurezza è delegata a `JdbcUserDetailsManager`, che opera sulle tabelle `USERS` e `AUTHORITIES`.
- **View.** I template Thymeleaf compongono le pagine HTML. I componenti comuni, intestazione, barra di navigazione, carosello delle recensioni, footer e script Bootstrap, sono estratti come fragment riutilizzabili nella directory `templates/bases/`. Il frontend usa Bootstrap 5 per il layout e Chart.js per gli istogrammi delle statistiche; le chiamate alle recensioni usano `fetch` con `async/await` per aggiornare la pagina senza ricaricarla. Abbiamo scelto `async/await` perché rispetto a `XMLHttpRequest` o alle Promise con `.then()` il codice risulta più leggibile e segue il flusso di esecuzione in modo più lineare.
- **Controller.** I controller gestiscono le richieste HTTP seguendo il pattern **controller** → **service** → **repository** e sono organizzati per area funzionale (`MainController`, `AdminDashboardController`, `TrainingController`, `ReviewController`, ...). I service incapsulano la logica applicativa: `CheckUserService` verifica l'unicità dello username, `PlanService` valida i piani con una whitelist dei ruoli consentiti per impedire la *privilege escalation* ad amministratore, `PasswordValidationService` valida il formato della password lato server, necessario perché le validazioni JavaScript lato client possono essere aggirate modificando le richieste HTTP.

2.2. Comunicazione tra i servizi

La web app principale comunica con il REST service tramite OpenFeign: il client `TrainingsProxy` dichiara gli endpoint del REST service come metodi Java annotati, e il framework genera l'implementazione HTTP. Il REST service espone i programmi predefiniti, la composizione di ciascun programma e il calcolo delle calorie.

2.3. Diagramma dell'architettura

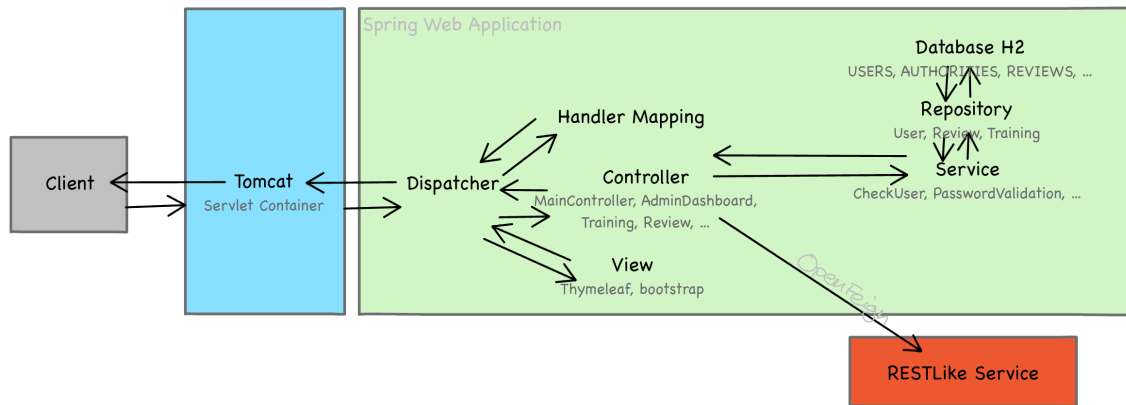


Figura 1. Diagramma dell'architettura del sistema.

3. Funzionalità

3.1. Descrizione della funzionalità selezionata

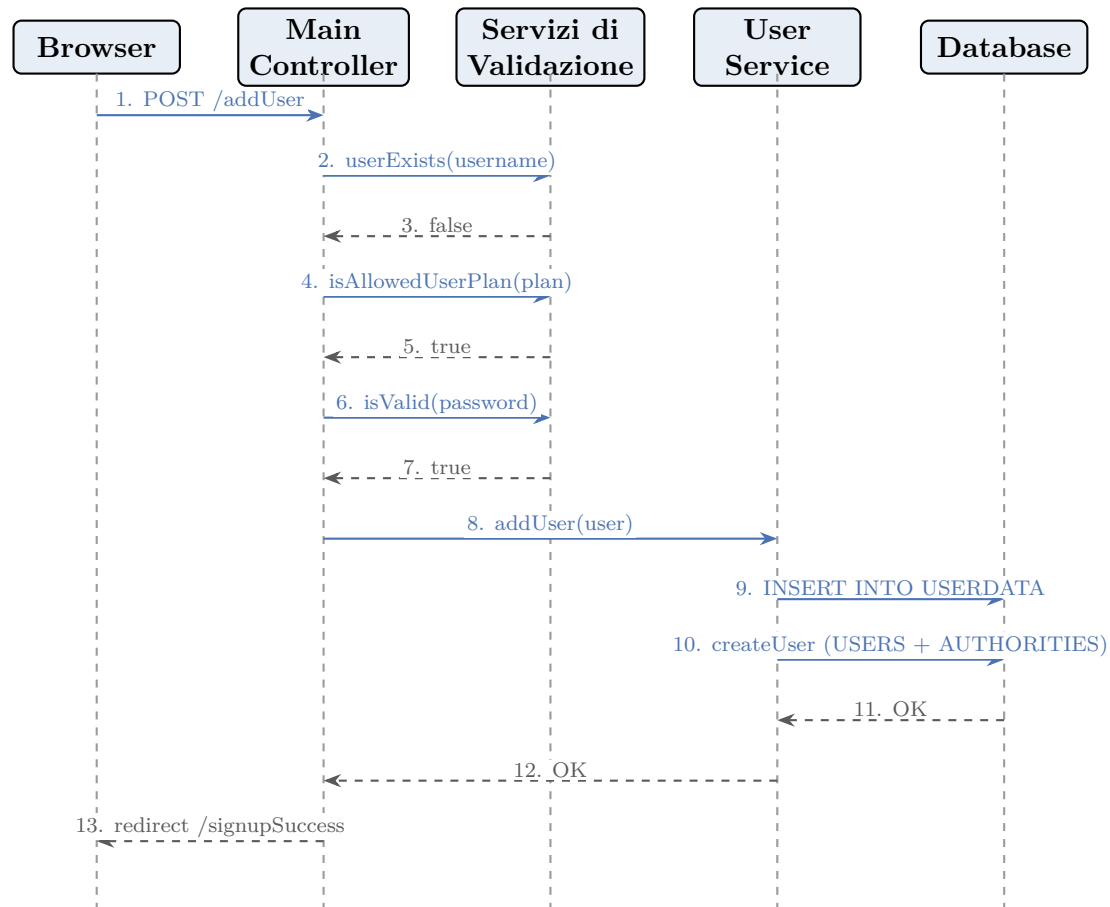
La funzionalità descritta è la **registrazione di un nuovo utente** (*signup*), scelta perché tocca più strati del sistema: validazione JavaScript lato client, più service layer distinti sul backend, creazione di record in più tabelle e gestione degli errori.

3.2. Interazione utente

Dal lato utente, i passi sono i seguenti:

1. L'utente non autenticato accede a `/signup`, dove trova un form con nome, cognome, data di nascita, email, username e password, più un set di card con radio button per la selezione del piano (Prova, Basic o Pro).
2. Prima dell'invio, JavaScript valida il formato della data di nascita, la maggiore età e il formato della password lato client.
3. Il form invia una richiesta `POST /addUser` al server.
4. Se lo username è già in uso, il backend restituisce un errore e ripropone il form.
5. Il backend ripete la validazione della password e verifica che il piano selezionato sia consentito. In caso di irregolarità restituisce un errore.
6. Se tutti i controlli hanno esito positivo, l'utente viene creato e viene mostrata la pagina di conferma.

3.3. Flusso interno al sistema



4. Criticità della soluzione e sviluppi futuri

4.1. Problemi incontrati durante lo sviluppo

H2 persiste su disco tra un avvio e l'altro, quindi le modifiche allo schema non venivano applicate perché `CREATE TABLE IF NOT EXISTS` lasciava le tabelle esistenti. Abbiamo risolto aggiungendo `DROP TABLE` e `DELETE` all'inizio di `schema.sql` e `data.sql`. Secondo problema: Spring Security mantiene il ruolo nella sessione corrente, quindi aggiornare `AUTHORITIES` dopo un upgrade non ha effetto immediato; l'utente deve fare logout e login per attivarlo.

4.2. Limiti della soluzione attuale

- L'amministratore non ha un'interfaccia per aggiungere o modificare programmi predefiniti. Per farlo è necessario modificare i file SQL del REST manipolandone li database.
- Le statistiche sono contatori cumulativi, non log. Non ci sono timestamp per sessione, quindi non è possibile analizzare la frequenza degli allenamenti nel tempo.

- Non abbiamo integrato un gateway di pagamento reale né implementato i requisiti GDPR.
- Il form dei contatti non invia nulla al centro.
- H2 embedded non è adatto alla produzione: poche connessioni concorrenti e persistenza legata al filesystem locale.

4.3. Possibili miglioramenti futuri

- Sostituire H2 con PostgreSQL o MySQL.
- Collegare un gateway di pagamento e implementare la conformità GDPR.
- Aggiungere timestamp alle sessioni completate per abilitare statistiche temporali.

5. Contributo di ogni componente del gruppo

Componente	Attività svolte
Stefano Videsott	Architettura iniziale della web app principale; Spring Security, controller, service layer e repository; parte del frontend frontend (relativo alle parti sviluppate), autenticazione e registrazione, dahsboards con alcune delle funzionalità, logica legata alle recensioni con <code>fetch/async/await</code> ; statistiche Chart.js; refactoring completo finale della codebase per uniformare i contributi di entrambi i membri (nomi di classi, metodi, variabili, tabelle e colonne DB rinominati in inglese con convenzioni coerenti).
Alessandro Como	Realizzazione della homepage e della pagina di contatto. Progettazione e sviluppo del REST service: controller, repository, schema SQL e dati di test; integrazione con OpenFeign REST; buisness logic e view relative alla visualizzazione e al completamento degli allenamenti e delle parti collegate.

La prima parte di ideazione e modellazione dello schema del database è stata fatta insieme.

6. Lezioni apprese

6.1. Aspetti tecnici

Abbiamo innanzitutto imparato le basi di Spring, un framework che nessuno di noi due aveva mai utilizzato; abbiamo imparato ad impostare correttamente la parte di security per gestire correttamente authorization e authentication. Abbiamo inoltre

consolidato concetti della programmazione asincrona, che avevamo già visto nel corso di *Ingegneria del Software* e che in questo corso abbiamo capito davvero.

6.2. Aspetti progettuali

Sviluppare in parallelo sulla stessa applicazione ci ha permesso, al termine del lavoro, di renderci conto che è utile concordare dei pattern di sviluppo e delle convenzioni di naming, senza le quali ci si trova ad avere del codice eterogeneo e dispersivo.

6.3. Cosa rifaremmo diversamente

Avremmo definito lo schema del database e le convenzioni di naming prima di iniziare: ogni modifica a progetto completato ha richiesto tempo per la comprensione e adeguare il codice in modo uniforme, operazione ripetuta troppe volte nella fase di refactor che ha richiesto diverse ore. Per il frontend useremmo una libreria differente per avere un frontend meno vincolato dalla logica dei dati oppure quantomeno Tailwind CSS al posto di Bootstrap, per avere un look più moderno e facile da scrivere.

6.4. Risorse aggiuntive

Nel corso del progetto sono state consultate diverse risorse esterne, in particolare per la realizzazione del frontend. Le principali sono:

- Baeldung, textitthymeleaf-extras-springsecurity6: <https://www.baeldung.com/spring-security-thymeleaf>
- Documentazione ufficiale Bootstrap 5: componenti carousel, navbar, tabelle, alert, radio toggle buttons, layout dei form e utility di sizing e spacing @ <https://getbootstrap.com/docs/5.3/>
- W3Schools, Bootstrap navbar e form @ https://www.w3schools.com/bootstrap5/bootstrap_navbar.php, https://www.w3schools.com/bootstrap/bootstrap_forms.asp
- Colorlib, template login di riferimento per il layout delle pagine di accesso @ <https://colorlib.com/wp/template/login-form-11/>
- MDN Web Docs, template literals JavaScript @ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals
- Stack Overflow, formattazione decimali in Thymeleaf e centratura elementi con Bootstrap @ <https://stackoverflow.com/questions/52132580>, <https://stackoverflow.com/questions/42388989>
- chart.js @ <https://www.chartjs.org/>